Lab 5: Static Code Analysis

Gym Membership System

Name: Krishna Jha
SRN: PES1UG24CS236
Assignment: 48 (spreadsheet row 49)

Final results

Pylint: **10.00/10** (before: 4.81/10).
Bandit: **0 findings** (before: 6).
Flake8: **0 findings** (before: 9).
Regression suite: **10 tests passed**.

The submission contains the cleaned Python program, before and after reports, an issue table, and reflection answers. Three local commits record the fixes. No analysis checks were suppressed.

Assigned source: github.com/SETAPESU26/L5_48_gym_membership_system
Source commit: 1bbe424240e3a8f7dc099ff1782f7b6a15c6f913

Run pylint, bandit, and flake8 on gym_membership_system.py to verify the cleaned application. Raw evidence is in reports/before and reports/after.

Lab 5: Identified issues and fixes

Name: Krishna Jha **SRN:** PES1UG24CS236 **Assigned program:** Gym Membership System (assignment 48, spreadsheet row 49)

Line numbers below refer to the original assigned script before fixes. Bandit severity is reproduced exactly. For Pylint/Flake8 findings, priorities are review judgments, since these tools do not assign Bandit's severity levels.

Issue	Type	Line(s)	Description	Fix Approach
B605: shell invocation	Security — High (Bandit)	58	User-controlled filename is concatenated into `os.system()`, allowing shell metacharacters to execute commands.	Replace the shell call with `print()`. File export uses Python file I/O only.
B324: MD5	Security — High (Bandit)	47	MD5 is a weak digest when used for security.	Use SHA-256 with explicit UTF-8 encoding. The result is an identifier digest, not password storage or authentication.
B307 / W0123: `eval()`	Security — Medium (Bandit)	40–43	An arbitrary discount expression can execute Python code.	Replace expressions with a numeric discount fraction between 0 and 1. Compute `fee * (1 - discount_rate)` directly; reject invalid ranges. Update the example to `0.15`, preserving its 510.0 result.
B105: embedded portal secret	Security — Low (Bandit)	70, 79	An unused demonstration credential is hardcoded and printed.	Remove the credential and its output entirely; the demo has no portal integration requiring a secret.
B311: predictable PIN source	Security — Low (Bandit)	51	`random.randint()` is unsuitable for generating a locker credential.	Use `1000 + secrets.randbelow(9000)`, preserving the four-digit range with a cryptographic random source.
B110 / W0702 / E722: silent bare exception	Reliability — Low (Bandit); Medium review priority	24–27	A bare `except: pass` hides all failures during cancellation.	Use `members.pop(m_id, None)` and return whether a member existed. Unexpected errors propagate.
W0102: mutable default argument	Bug — Medium review priority	15, 19	The same default list persists across registrations, retaining hidden state.	Default to `None` and create a new list per call. Preserve an explicitly supplied caller log.
W1514 / R1732: file lifecycle	Reliability — Medium review priority	55–57	Manual closure can be skipped if writing fails; encoding depends on the environment.	Use a `with open(..., encoding="utf-8")` context manager so closure occurs on errors too.
C0121 / E711: comparison to `None`	Quality/style — Low review priority	32	Equality comparison is inappropriate for a singleton.	Look up the member once and test `member is not None`.
R1703 / R1705: redundant conditionals	Maintainability — Low review priority	34–37	Explicit True/False branches and an `else` after return add unnecessary control flow.	Return the membership predicate directly.
W0621 / W0612: shadowing and unused loop key	Maintainability — Low review priority	61, 63	A parameter shadows the global roster, and the iteration key is unused.	Rename the parameter to `roster` and sum active predicates over `roster.values()`.
W0611 / F401: unused import	Quality — Low review priority	6	`math` is never used.	Remove it; also remove `os` after eliminating shell execution.

Issue	Type	Line(s)	Description	Fix Approach
W0612 / F841: unused variables	Quality — Low review priority	31, 41	`locker_deposit` is unused; `base_fee` appears unused because its use was hidden inside an evaluated string.	Remove the deposit variable and use the member fee directly in arithmetic.
C0103: function naming	Maintainability — Low review priority	15, 23, 30, 40	CamelCase function names violate Python conventions.	Rename to snake_case and update all internal callers and tests.
C0116 / C0209: documentation and formatting	Maintainability — Low review priority	15–69, 19	Functions lack docstrings and registration uses older string interpolation.	Document function contracts and use an f-string for log entries.
E501 / W292 / C0304: line length and newline	Style — Low review priority	10, 11, 18, 82	Dictionary lines exceed 79 characters and the final newline is missing.	Wrap dictionaries and add the final newline. Keep standard two-blank-line function separation.
Unconditional demo execution (manual review)	Maintainability — Medium review priority	82	Importing the module changes the roster, writes a file, and prints output.	Add an `if __name__ == "__main__":` guard so imports have no demo side effects. This was not a tool-reported finding.

Verification

Check	Before	After
Pylint	4.81/10; 28 messages	10.00/10; no messages
Bandit	6 findings: 2 High, 1 Medium, 3 Low	0 findings; no skipped checks
Flake8	9 findings	0 findings
Regression tests	No supplied test suite	10 tests pass

Reports in `reports/before/` analyze the original assigned script. Reports in `reports/after/` analyze only the cleaned application, using the same default tool rules. No warnings were disabled, no `nosec` markers were added, and no line-length limits were relaxed. Pylint uses `--persistent=n` only to avoid writing its cache outside the project. The empty final Flake8 file means no findings. All final tool runs completed successfully before submission cleanup.

Normal demo behavior is retained: M04 registers, M02 is cancelled, M01 is active, renewal costs 510.0, the roster is exported, and the active count is 3. Intentional interface changes are the numeric discount argument, snake_case function names, a SHA-256 digest instead of MD5, and removal of secret output.

Lab 5: Reflection

Name: Krishna Jha **SRN:** PES1UG24CS236

The assigned repository requests four reflection answers, but the supplied four-page handout does not include the actual reflection questions. The four questions below are therefore explicitly chosen to cover the lab's learning outcomes; they are not quoted from the handout.

1. How did the three static analysis tools complement each other?

Pylint identified quality and maintainability problems such as a shared default list, missing docstrings, unused variables, and resource handling. Flake8 made the code consistent by reporting long lines, a bare exception, an inappropriate comparison to None, and a missing final newline. Bandit identified security patterns such as eval, shell execution, MD5, insecure randomness, and an embedded secret. Their reports overlapped in places, but each contributed useful checks.

2. Which issues were prioritized, and why?

The two High Bandit findings and the Medium eval finding came first. Shell execution could interpret a filename as commands, and eval could execute a discount expression as Python. These execution paths were removed completely. MD5 was replaced with SHA-256. The Low Bandit findings were also resolved by removing the unused credential, using secrets for PIN generation, and replacing silent exception handling. The shared default list was then fixed because its hidden state could produce confusing behavior across registrations.

3. What required judgment beyond following a tool's suggestion?

Replacing eval required deciding what the function should accept. A simple numeric discount fraction expresses the actual business operation without supporting a programming language. The sample's 15% discount still gives 510.0. Merely substituting ast.literal_eval would not evaluate the original arithmetic expression involving base_fee. The hardcoded secret was removed because no portal client uses it; moving an unnecessary secret into an environment variable would add configuration without serving the program. Manual review also found that the unguarded main call performed work on import. The README described an input prompt, but the actual supplied program has none; its real entry point, rather than that description, guided the smoke run.

4. How were the fixes verified, and what are static analysis's limits?

The same three tools were rerun on the cleaned application. Pylint improved from 4.81/10 to 10.00/10, all six Bandit findings disappeared, and Flake8 went from nine findings to none. Ten regression tests verified membership operations, discount boundaries and rejection of expressions, PIN range, safe export with shell metacharacters in the filename, import behavior, and the demonstration. These results give evidence for the fixes, but a clean report does not prove the application is secure or its business rules are correct. A four-digit PIN still needs attempt limiting in a real locker system, and an identifier digest is not an authentication mechanism. Production validation, persistence, and access control would need their own requirements and tests.